

TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA



TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero - Junio 2026

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de diseño de software

TÍTULO DE LA ACTIVIDAD:
Patrón de diseño Composite

UNIDAD A EVALUAR:
3

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO

Roberto Chavez Moreno 22210292

NOMBRE DEL MAESTRO (A):

Maribel Guerrero Luis

Introducción

En el desarrollo de software, los patrones de diseño juegan un papel fundamental al proporcionar soluciones reutilizables a problemas comunes. Uno de estos patrones es el patrón Composite, el cual pertenece a la categoría de patrones estructurales y permite organizar objetos en estructuras jerárquicas tipo árbol.

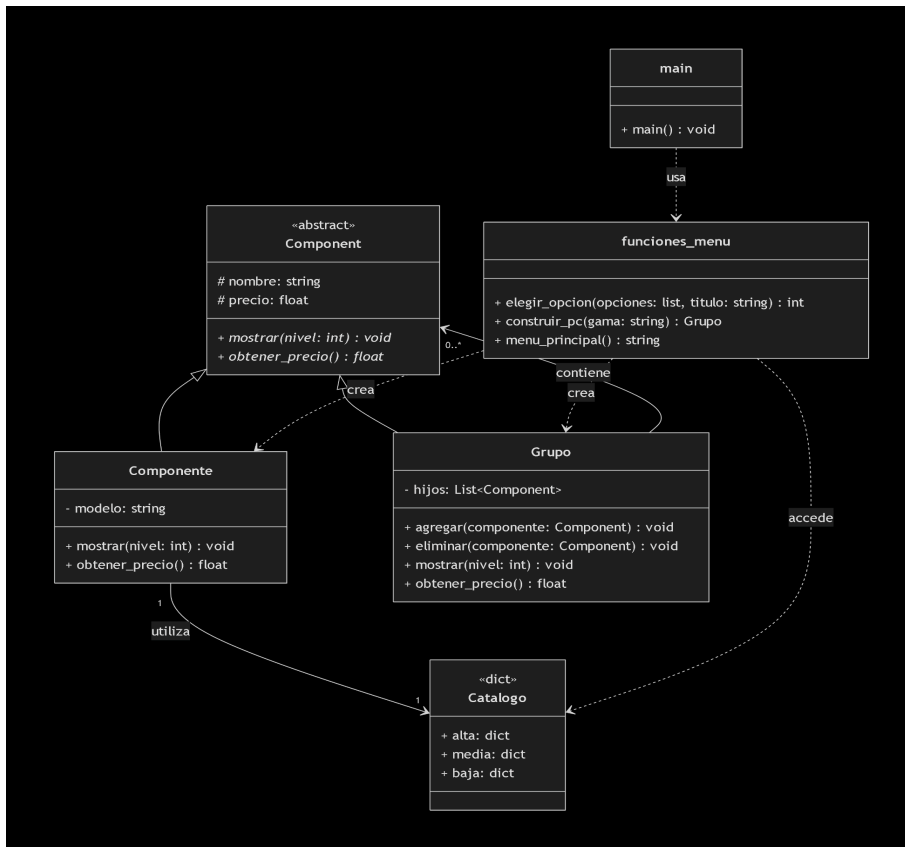
El objetivo principal del patrón Composite es permitir que los clientes puedan tratar de manera uniforme tanto a los objetos individuales como a las composiciones de objetos, sin necesidad de distinguir entre ellos. Esta característica facilita el manejo de estructuras complejas, ya que todos los elementos comparten una misma interfaz .

Este patrón se basa en una estructura jerárquica en forma de árbol, donde existen tres elementos principales:

- **Component:** Define la interfaz común para todos los objetos.
- **Leaf (Hoja):** Representa objetos individuales sin hijos.
- **Composite (Compuesto):** Contiene otros objetos (Leaf o Composite).

En esta práctica, se implementa este patrón para simular un constructor de computadoras, donde cada componente de hardware es un objeto individual, y la PC completa es una composición de dichos componentes.

Diagrama UML



```

from abc import ABC, abstractmethod

# -----
# 1. COMPONENT - interfaz abstracta comun
# -----
class Component(ABC):
    """Clase abstracta para todos los nodos del arbol."""

    def __init__(self, nombre: str, precio: float = 0.0):
        self.nombre = nombre
        self.precio = precio

    @abstractmethod
    def mostrar(self, nivel: int = 0) -> None:
        """Muestra el componente con sangria segun su nivel."""
        pass

    @abstractmethod
    def obtener_precio(self) -> float:
        """Retorna el precio del componente o subtotal."""
        pass

# -----
# 2. LEAF - componente individual (nodo hoja)

```

```

# -----
class Componente(Component):
    """
    representa un componente real de hardware
    (CPU, GPU, RAM, Almacenamiento, Fuente de Poder, Gabinete).
    muestra su informacion y precio, y retorna su precio.
    """

    def __init__(self, nombre: str, modelo: str, precio: float):
        super().__init__(nombre, precio)
        self.modelo = modelo

    def mostrar(self, nivel: int = 0) -> None:
        sangria = "    " * nivel
        precio_str = f"MXN ${self.precio:,.2f}" if self.precio > 0 else "Incluido"
        print(f"{sangria}[+] {self.nombre}: {self.modelo} -> {precio_str}")

    def obtener_precio(self) -> float:
        return self.precio

# -----
# 3. COMPOSITE - agrupacion de componentes
# -----
class Grupo(Component):
    """
    Nodo compuesto: puede contener Leafs u otros Composites.
    Representa la PC completa como raiz del arbol.
    """

    def __init__(self, nombre: str):
        super().__init__(nombre, 0.0)
        self._hijos: list[Component] = []

    def agregar(self, componente: Component) -> None:
        """Agrega un hijo (Leaf o Composite) al grupo."""
        self._hijos.append(componente)

    def eliminar(self, componente: Component) -> None:
        """Elimina un hijo del grupo."""
        self._hijos.remove(componente)

    def mostrar(self, nivel: int = 0) -> None:
        sangria = "    " * nivel
        prefijo = "[PC]" if nivel == 0 else "[--]"
        print(f"{sangria}{prefijo} {self.nombre}")
        for hijo in self._hijos:
            hijo.mostrar(nivel + 1) # recursion -> recorre el arbol

```

```

def obtener_precio(self) -> float:
    # Suma recursiva de todos los hijos
    return sum(hijo.obtener_precio() for hijo in self._hijos)

# -----
# 4. CATALOGO DE COMPONENTES POR GAMA
# -----

CATALOGO = {
    # =====
    # GAMA ALTA - gaming profesional / workstation
    # =====
    "alta": {

        # Procesadores de alto rendimiento
        "CPU": [
            ("AMD Ryzen 9 7950X ", 14_299.00),
            ("Intel Core i9-13900K ", 13_499.00),
            ("AMD Ryzen 9 7900X ", 10_999.00),
        ],

        # Tarjetas graficas tope de gama
        "GPU": [
            ("NVIDIA GeForce RTX 5090 ", 36_499.00),
            ("NVIDIA GeForce RTX 5080 ", 24_999.00),
            ("AMD Radeon RX 7900 XTX ", 22_499.00),
        ],

        # Memoria RAM de alta velocidad DDR5
        "RAM": [
            ("Corsair Dominator 32 GB DDR5 6000 MHz", 5_299.00),
            ("G.Skill Trident Z5 64 GB DDR5 5600 MHz", 9_999.00),
            ("Kingston Fury Beast 32 GB DDR5 5200 MHz", 4_799.00),
        ],

        # Almacenamiento NVMe ultrarapido
        "Almacenamiento": [
            ("Samsung 990 Pro SSD NVMe 2 TB Gen4", 6_499.00),
            ("WD Black SN850X SSD NVMe 2 TB Gen4", 5_999.00),
            ("Seagate FireCuda 530 SSD NVMe 4 TB Gen4", 11_499.00),
        ],

        # Fuentes de poder certificadas 80+ Gold / Platinum
        "Fuente de Poder": [
            ("Corsair RM1000x 1000W 80+ Gold Modular", 3_199.00),
            ("Seasonic FOCUS GX-850W 80+ Gold Modular", 2_699.00),
        ]
    }
}

```

```

        ("EVGA SuperNOVA 850W G6 80+ Gold Modular", 2_899.00),
    ],

    # Gabinetes full/mid tower con buena ventilacion
    "Gabinete": [
        ("Lian Li PC-011 Dynamic XL Full Tower ATX", 3_999.00),
        ("NZXT H9 Elite Mid Tower ATX Panel Vidrio", 3_499.00),
        ("Fractal Design Torrent Compact Mid Tower", 2_999.00),
    ],
},

# =====
# GAMA MEDIA - gaming casual / trabajo creativo
# =====
"media": {

    # Procesadores equilibrados
    "CPU": [
        ("AMD Ryzen 5 7600X ", 5_799.00),
        ("Intel Core i5-13600K ", 5_999.00),
        ("AMD Ryzen 7 5700X ", 4_799.00),
    ],

    # Tarjetas graficas de rendimiento medio
    "GPU": [
        ("NVIDIA GeForce RTX 5060 Ti ", 8_499.00),
        ("NVIDIA GeForce RTX 4060 Ti ", 6_999.00),
        ("AMD Radeon RX 6700 XT ", 7_299.00),
    ],

    # Memoria RAM DDR4 estandar
    "RAM": [
        ("Corsair Vengeance 16 GB DDR4 3600 MHz", 1_399.00),
        ("Kingston Fury Beast 32 GB DDR4 3200 MHz", 2_199.00),
        ("G.Skill Ripjaws V 16 GB DDR4 3200 MHz", 1_299.00),
    ],

    # Almacenamiento SSD asequible
    "Almacenamiento": [
        ("Kingston NV2 SSD NVMe 1 TB Gen4", 999.00),
        ("WD Blue SN580 SSD NVMe 1 TB Gen4", 1_149.00),
        ("Samsung 870 EVO SSD SATA 1 TB", 1_399.00),
    ],

    # Fuentes de poder certificadas 80+ Bronze
    "Fuente de Poder": [
        ("Corsair CV650 650W 80+ Bronze", 999.00),
        ("Thermaltake Smart BX1 650W 80+ Bronze", 899.00),
    ],
}

```

```

        ("EVGA 600W BR 80+ Bronze Semi-Modular",      849.00),
    ],

    # Gabinetes mid tower accesibles
    "Gabinete": [
        ("Cooler Master MasterBox MB520 Mid Tower",    1_599.00),
        ("NZXT H510 Flow Mid Tower ATX",               1_499.00),
        ("Deepcool CC560 Mid Tower 4 Ventiladores",    999.00),
    ],
},

# =====
# GAMA BAJA - ofimatica / uso basico / escolar
# =====
"baja": {

    # Procesadores economicos
    "CPU": [
        ("AMD Ryzen 3 4100 ",      1_799.00),
        ("Intel Core i3-12100F ",  1_899.00),
        ("AMD Athlon 3000G ",      999.00),
    ],

    # Opciones graficas basicas
    "GPU": [
        ("NVIDIA GeForce GT 1030 ",  1_499.00),
        ("AMD Radeon RX 550 ",       1_299.00),
        ("Graficos integrados iGPU (sin costo)",    0.00),
    ],

    # RAM basica DDR4
    "RAM": [
        ("Kingston ValueRAM 8 GB DDR4 2666 MHz",    549.00),
        ("Crucial 16 GB DDR4 2400 MHz",             1_049.00),
        ("Patriot Signature 8 GB DDR4 3200 MHz",    599.00),
    ],

    # Almacenamiento HDD economico
    "Almacenamiento": [
        ("Seagate Barracuda HDD 1 TB 7200 rpm",     849.00),
        ("Western Digital Blue HDD 2 TB 5400 rpm",  1_199.00),
        ("Kingston A400 SSD SATA 480 GB",          699.00),
    ],

    # Fuentes de poder basicas
    "Fuente de Poder": [
        ("Thermaltake Smart 500W 80+ White",        649.00),
        ("Azza 500W 80+ White",                    499.00),
    ],
}

```

```

        ("Cougar VTE500 500W 80+ White", 579.00),
    ],

    # Gabinetes economicos
    "Gabinete": [
        ("Deepcool Matrexx 30 Mini Tower Micro-ATX", 599.00),
        ("Aerocool Cylon Mid Tower ATX", 549.00),
        ("Cougar MX330-X Mid Tower ATX", 699.00),
    ],
},
}

# -----
# 5. FUNCIONES DE MENU (interfaz de consola)
# -----

def elegir_opcion(opciones: list, titulo: str) -> int:
    """Muestra un menu numerado y valida que se ingrese un numero valido."""
    print(f"\n {titulo}")
    print(" " + "-" * 55)
    for i, opcion in enumerate(opciones, 1):
        nombre, precio = opcion
        if precio > 0:
            print(f" [{i}] {nombre}")
            print(f" Precio: MXN ${precio:,.2f}")
        else:
            print(f" [{i}] {nombre}")
            print(f" Precio: Incluido")
    print(" " + "-" * 55)

    while True:
        try:
            eleccion = int(input("\n Tu eleccion: "))
            if 1 <= eleccion <= len(opciones):
                return eleccion - 1
            print(" AVISO: Numero fuera de rango, intenta de nuevo.")
        except ValueError:
            print(" AVISO: Ingresa un numero valido.")

def construir_pc(gama: str) -> Grupo:
    """
    Guia al usuario a elegir componente por componente
    y construye el arbol Composite de la PC.
    """
    pc = Grupo(f"PC Gama {gama.upper()}") # Composite raiz

```



```

for categoria, opciones in CATALOGO[gama].items():
    indice = elegir_opcion(opciones, f"Elige {categoria}:")
    modelo, precio = opciones[indice]

    # Crear Leaf (nodo hoja) y agregarlo al Composite raiz
    componente = Componente(categoria, modelo, precio)
    pc.agregar(componente)

return pc

def menu_principal() -> str:
    """Muestra el menu de seleccion de gama."""
    print("\n" + "=" * 55)
    print("  CONSTRUCTOR DE PC  -  Patron de Diseno Composite")
    print("=" * 55)
    print("  [1] Gama Alta    -- Gaming Pro / Workstation")
    print("  [2] Gama Media   -- Gaming Casual / Trabajo Creativo")
    print("  [3] Gama Baja    -- Ofimatica / Uso Basico")
    print("  [0] Salir")
    print("=" * 55)

    opciones_validas = {"1": "alta", "2": "media", "3": "baja", "0": "salir"}
    while True:
        eleccion = input("\n  Elige una opcion: ").strip()
        if eleccion in opciones_validas:
            return opciones_validas[eleccion]
        print("  AVISO: Opcion no valida, ingresa 0, 1, 2 o 3.")

# -----
# 6. PROGRAMA PRINCIPAL
# -----

def main():
    print("\n  Bienvenido al Constructor de PC con Patron Composite")

    while True:
        gama = menu_principal()

        if gama == "salir":
            print("\n  Hasta luego!\n")
            break

        # Construir el arbol de componentes (Composite)
        pc = construir_pc(gama)

        # Mostrar la configuracion final en forma de arbol

```

```
print("\n" + "=" * 55)
print("  CONFIGURACION FINAL  (estructura de arbol)")
print("=" * 55)
pc.mostrar()

# Calcular y mostrar precio total (suma recursiva)
total = pc.obtener_precio()
print("\n" + "-" * 55)
print(f"  PRECIO TOTAL: MXN ${total:,.2f}")
print("-" * 55)

otra = input("\n  Construir otra PC? (s/n): ").strip().lower()
if otra != "s":
    print("\n  Hasta luego!\n")
    break

if __name__ == "__main__":
    main()
```

Bienvenido al Constructor de PC con Patron Composite

=====

CONSTRUCTOR DE PC - Patron de Diseno Composite

=====

- [1] Gama Alta -- Gaming Pro / Workstation
 - [2] Gama Media -- Gaming Casual / Trabajo Creativo
 - [3] Gama Baja -- Ofimatica / Uso Basico
 - [0] Salir
- =====

Elige una opcion: 1

Elige CPU:

-
- [1] AMD Ryzen 9 7950X
Precio: MXN \$14,299.00
 - [2] Intel Core i9-13900K
Precio: MXN \$13,499.00
 - [3] AMD Ryzen 9 7900X
Precio: MXN \$10,999.00
-

Tu eleccion: 2

Elige GPU:

-
- [1] NVIDIA GeForce RTX 5090
Precio: MXN \$36,499.00
 - [2] NVIDIA GeForce RTX 5080
Precio: MXN \$24,999.00
 - [3] AMD Radeon RX 7900 XTX
Precio: MXN \$22,499.00
-

Tu eleccion: 3

Elige RAM:

-
- [1] Corsair Dominator 32 GB DDR5 6000 MHz
Precio: MXN \$5,299.00
 - [2] G.Skill Trident Z5 64 GB DDR5 5600 MHz
Precio: MXN \$9,999.00
 - [3] Kingston Fury Beast 32 GB DDR5 5200 MHz
Precio: MXN \$4,799.00
-

Elige Almacenamiento:

- [1] Samsung 990 Pro SSD NVMe 2 TB Gen4
Precio: MXN \$6,499.00
- [2] WD Black SN850X SSD NVMe 2 TB Gen4
Precio: MXN \$5,999.00
- [3] Seagate FireCuda 530 SSD NVMe 4 TB Gen4
Precio: MXN \$11,499.00

Tu eleccion: 2

Elige Fuente de Poder:

- [1] Corsair RM1000x 1000W 80+ Gold Modular
Precio: MXN \$3,199.00
- [2] Seasonic FOCUS GX-850W 80+ Gold Modular
Precio: MXN \$2,699.00
- [3] EVGA SuperNOVA 850W G6 80+ Gold Modular
Precio: MXN \$2,899.00

Tu eleccion: 1

Elige Gabinete:

- [1] Lian Li PC-O11 Dynamic XL Full Tower ATX
Precio: MXN \$3,999.00
- [2] NZXT H9 Elite Mid Tower ATX Panel Vidrio
Precio: MXN \$3,499.00
- [3] Fractal Design Torrent Compact Mid Tower
Precio: MXN \$2,999.00

Tu eleccion: 1

=====

CONFIGURACION FINAL (estructura de arbol)

=====

[PC] PC Gama ALTA

- [+] CPU: Intel Core i9-13900K -> MXN \$13,499.00
- [+] GPU: AMD Radeon RX 7900 XTX -> MXN \$22,499.00
- [+] RAM: Kingston Fury Beast 32 GB DDR5 5200 MHz -> MXN \$4,799.00
- [+] Almacenamiento: WD Black SN850X SSD NVMe 2 TB Gen4 -> MXN \$5,999.00
- [+] Fuente de Poder: Corsair RM1000x 1000W 80+ Gold Modular -> MXN \$3,199.00
- [+] Gabinete: Lian Li PC-O11 Dynamic XL Full Tower ATX -> MXN \$3,999.00

PRECIO TOTAL: MXN \$53,994.00

Conclusiones

En la actividad se pudo implementar el patrón composite para armar computadoras de diferentes gamas a partir de diferentes componentes, el patrón es útil en estructuras tipo árbol, como en este caso, donde una computadora está formada por múltiples componentes.

Referencias

<https://refactoring.guru/design-patterns/composite>

<https://www.geeksforgeeks.org/java/composite-design-pattern-in-java/#components-of-composite-design-pattern>